

网络安全创新创业实践

7 月 20 日作业一

Bitcoin 测试网络交易发送、逐字节解析
与完整区块验证

姓名： _____

学号： _____

班级： _____

完成日期： 2026 年 7 月 20 日

摘要

本作业面向 Bitcoin 测试网络,完成了两笔真实 Testnet4 交易的构造与广播,并从线协议字节出发独立实现交易、区块和 Script 的解析与验证。第一笔交易花费 5 个锁定为 P2WSH(SHA256(OP_TRUE)) 的公开教学输出,将 1650 sat 合并为课程钱包的 1000 sat 输出;第二笔交易由课程专用私钥按 BIP143 构造签名原像,使用 RFC6979 确定性 nonce 和 low-S ECDSA 签名,将 400 sat 发往接收地址,并产生 318 sat 找零。两个广播交易的 txid 分别为 4aa330ae...25f7a 和 ad00f80a...9ac105。

实现未调用第三方 Bitcoin 解析库,而是使用 Python 标准库完成 CompactSize、大小端转换、SegWit 序列化、txid/wtxid、Merkle 树、紧凑难度目标、secp256k1 点运算、DER 签名和 P2WPKH/P2WSH Script 审计。课程签名交易的 223 个字节以及包含父交易的已确认完整区块的 4045 个字节均被逐字节映射到唯一协议字段;ECDSA、CleanStack、Merkle 根和 PoW 独立重算全部通过。父交易已在高度 144878 确认,子交易在最终采样时仍为 mempool 未确认状态。另以一笔已确认默认 Signet P2WPKH 交易和其 74123 字节区块作为回归样本。报告给出题目原文、理论、完整计算、运行结果、图形证据、测试和全部源代码。

关键词: Bitcoin; Testnet4; SegWit; P2WPKH; P2WSH; BIP143; ECDSA; Merkle 树; 逐字节解析

目录

1	作业原文、目标与完成边界	1
1.1	原始 Project 要求	1
1.2	网络选择与诚实性说明	2
2	规范与数学基础	2
2.1	UTXO 与交易序列化	2
2.2	Bech32 与 witness program	2
2.3	P2WSH(OP_TRUE) 的公开授权	3
2.4	BIP143 签名原像	3
2.5	secp256k1 ECDSA	3
2.6	区块头、目标值与 Merkle 根	4
3	设计与实现	4
3.1	工程结构	4
3.2	防错约束	5
4	真实交易构造与广播	5
4.1	课程地址	5

4.2	父交易: P2WSH(OP_TRUE) 合并	6
4.3	子交易: 课程私钥 P2WPKH 签名	6
5	课程签名交易逐字节解析	8
5.1	字段级完整表	8
5.2	位级展开示例与完整产物	10
6	BIP143 与 Script 严格验证	10
6.1	签名中间值	10
6.2	P2WPKH 栈执行	11
7	完整区块解析与 PoW 验证	12
8	测试、可复现性与结果讨论	13
8.1	自动化测试	13
8.2	复现实验命令	14
8.3	威胁模型与局限	14
9	结论	15
A	完整逐字节结果文件索引	16
B	核心源代码	16
B.1	bitcoin_codec.py	16
B.2	wallet_tool.py	27
B.3	claim_op_true_utxos.py	31
B.4	chain_analyzer.py	33
B.5	visualize_results.py	36
B.6	test_bitcoin_codec.py	39

1 作业原文、目标与完成边界

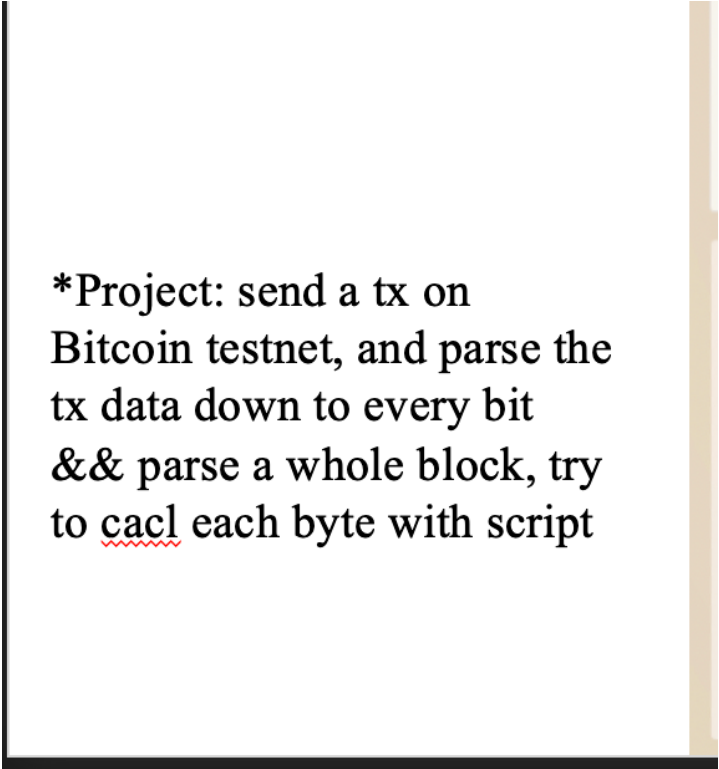
1.1 原始 Project 要求

课程截图中的 Project 原文为：

Project: send a tx on Bitcoin testnet, and parse the tx data down to every bit && parse a whole block, try to calc each byte with script.

结合课件同页的 Blockchain、Block、Tx/Coinbase tx、Input、Output、Locking Script、Unlocking Script 与 PoW 上下文，本报告将要求严格化为以下可验收目标：

1. 在公开 Bitcoin 测试网络广播至少一笔由本实验构造的交易，并保留 raw transaction、txid、节点返回和时间证据；
2. 不依赖浏览器的解码结果，从原始字节解析交易，给每个字节标注十六进制、8 位二进制和所属字段；
3. 从 80 字节区块头到最后一笔交易完整解析一个区块，重算区块哈希、目标值、PoW 与 Merkle 根，并证明无遗漏或重叠字节；
4. 对锁定/解锁条件逐步执行 Script，至少覆盖本实验的 P2WSH OP_TRUE 与 P2WPKH ECDSA 两条路径；
5. 附代码、运行结果、截图/可视化、自动化测试与引用资料。



***Project: send a tx on
Bitcoin testnet, and parse the
tx data down to every bit
&& parse a whole block, try
to cacl each byte with script**

图 1：课程截图中作业一的 Project 原文（裁剪自原始图片）

1.2 网络选择与诚实性说明

实验选择 **Bitcoin Testnet4**。BIP94 将 Testnet4 定义为替代 Testnet3 的新测试网络，并说明它除少量测试网难度规则外与主网共用共识规则，且从创世块起启用截至 2024 年 5 月主网已有软分叉 [1]。因此它比长期运行且存在 block storm 问题的 Testnet3 更适合作为本课程实验环境。

本实验先后尝试默认 Signet 水龙头；服务端接受排队请求但未产生 UTXO。为避免把离线交易冒充广播交易，最终采用 Testnet4 上锁定脚本明确授权任何人花费的 P2WSH (OP_TRUE) 教学输出作为启动资金。该行为不使用、猜测或泄露任何第三方私钥；授权完全来自链上 Script。随后课程钱包使用自己的 ECDSA 私钥花费收到的输出。报告只将节点明确返回 txid 的交易称为“已广播”；在采样时尚未入块的交易标注为“mempool 已接受、未确认”。

2 规范与数学基础

2.1 UTXO 与交易序列化

Bitcoin 交易不直接修改账户余额，而是引用既有未花费交易输出 (UTXO)，并创建新输出。输入通过

$$\text{outpoint} = \text{prev_txid}_{32\text{B}} \parallel \text{vout}_{4\text{B}}$$

定位被花费输出。金额为 8 字节小端无符号整数，单位 satoshi。输入数、输出数、脚本长度和 witness 项数使用 CompactSize：小于 0xfd 时占 1 字节，否则由前缀 fd/fe/ff 指示 2/4/8 字节小端载荷。实现额外拒绝非最短编码，避免同值多编码造成歧义。

BIP141 对含 witness 的交易定义 [2]：

$$\begin{aligned} \text{wire} = & [\text{version}][00][01][\text{vin}][\text{vout}] \\ & [\text{witness per input}][\text{locktime}]. \end{aligned}$$

其中 txid 仍为传统（不含 marker、flag、witness）序列化的双 SHA-256，而 wtxid 为完整 witness 序列化的双 SHA-256；哈希在常见显示形式中反转字节序：

$$\text{txid} = \text{rev}(\text{SHA256d}(\text{ser}_{\text{stripped}}(tx))), \quad \text{wtxid} = \text{rev}(\text{SHA256d}(\text{ser}_{\text{witness}}(tx))).$$

交易权重和虚拟大小为

$$W = 4S_{\text{stripped}} + (S_{\text{total}} - S_{\text{stripped}}), \quad \text{vsize} = \left\lceil \frac{W}{4} \right\rceil.$$

2.2 Bech32 与 witness program

BIP173 规定测试网 SegWit 地址的人类可读前缀为 **tb**，数据部分先把 witness program 从 8 位组转换为 5 位组，再附加 BCH 类校验码 [4]。本实验使用 witness v0：

- P2WPKH: 00 14 <HASH160(pubkey)>, program 长度 20 字节;
- P2WSH: 00 20 <SHA256(witnessScript)>, program 长度 32 字节。

2.3 P2WSH(OP_TRUE) 的公开授权

本实验启动资金的 witnessScript 仅为 51, 即 OP_TRUE。其 SHA-256 为

4ae81572f06e1b88fd5ced7a1a000945432e83e1551e6f721ee9c00b8cc33260.

因此锁定脚本是

00204ae81572f06e1b88fd5ced7a1a000945432e83e1551e6f721ee9c00b8cc33260.

BIP141 要求 P2WSH 的 witness 最后一项的 SHA-256 等于 32 字节 program, 然后执行该 witnessScript, 最终栈必须恰有一个 TRUE[2]。本例 witness 为 [51], 哈希相等后执行 OP_TRUE, 最终栈为 [01]。这就是链上显式的“任何人可花费”授权。

2.4 BIP143 签名原像

P2WPKH 的实际 scriptCode 为传统 P2PKH 模板

76 a9 14 <20-byte-pubkey-hash> 88 ac,

即 OP_DUP OP_HASH160 PUSH20 OP_EQUALVERIFY OP_CHECKSIG。对 SIGHASH_ALL, BIP143 的原像为 [3]:

$nVersion || hashPrevouts || hashSequence || outpoint || scriptCode$
 $|| amount || nSequence || hashOutputs || nLockTime || sighashType.$

其中三个聚合哈希均为双 SHA-256。与旧算法相比, BIP143 将被花费输出的金额纳入承诺, 并通过聚合哈希避免多输入交易签名验证的二次复杂度。

2.5 secp256k1 ECDSA

曲线和基点阶为

$$E: y^2 = x^3 + 7 \pmod{p},$$

$$p = 2^{256} - 2^{32} - 977.$$

$n = \text{FFFEBAAEDCE6AF48A03BBFD25E8CD0364141}.$

私钥为 $d \in [1, n-1]$ ，公钥 $Q = dG$ 。对摘要整数 z ，选择 nonce k ，令

$$R = kG = (x_R, y_R), \quad r = x_R \bmod n, \quad s = k^{-1}(z + rd) \bmod n.$$

验证时计算

$$w = s^{-1}, \quad X = (zw)G + (rw)Q,$$

接受条件为 $X \neq \mathcal{O}$ 且 $x_X \bmod n = r$ 。实现采用 RFC6979 HMAC-SHA256 确定性 nonce，并令 $s \leftarrow \min(s, n-s)$ 满足 Bitcoin 的 low-S 规范，从而移除 ECDSA 的 $s \leftrightarrow n-s$ 可塑性。

2.6 区块头、目标值与 Merkle 根

区块原始数据由 80 字节头、交易数 CompactSize 和连续交易组成。头字段依次为 version、previous block hash、Merkle root、time、nBits、nonce。区块标识为

$$\text{blockhash} = \text{rev}(\text{SHA256d}(\text{header})).$$

若 nBits 的最高字节为指数 e ，低 23 位为系数 c ，目标值为

$$T = c \cdot 256^{e-3}, \quad \text{int}(\text{SHA256d}(\text{header})) \leq T.$$

Merkle 树叶子为 txid 的内部字节序；每层两两连接后双 SHA-256，奇数节点复制末项，直到得到根。程序把重算根与区块头中的 32 字节字段比较。

3 设计与实现

3.1 工程结构

代码位于作业根目录的 code/ 子目录，职责见表 1。

表 1: 作业一代码与产物结构

文件	职责
code/bitcoin_codec.py	secp256k1、RFC6979/low-S ECDSA、DER、Bech32、CompactSize、交易/区块解析、BIP143、P2WPKH/P2WSH Script
code/wallet_tool.py	测试网钱包生成、UTXO 查询、1 入 2 出 P2WPKH 构造、签名、广播与证据 JSON
code/claim_op_true_utxos.py	按链上公开条件花费 P2WSH(OP_TRUE) 输出，构造并广播启动交易
code/chain_analyzer.py	拉取 raw tx/raw block，逐字段反序列化，输出每字节 8 位映射、CSV/JSON，并独立验证
code/visualize_results.py	从 JSON/CSV 生成字节布局、Script 栈和运行汇总图
code/test_bitcoin_codec.py	6 项确定性回归测试
output/	raw hex、逐字节映射、字段表、区块交易索引、Script trace、广播证据和图片

3.2 防错约束

解析器不是“尽可能读”而是“精确消费”：任何越界、非最短 CompactSize、字段重叠、尾随字节或重序列化不一致都会抛出异常。对长度为 L 的 raw data，字节标注器建立 L 个初始为 UNCLAIMED 的槽位；每个 Span 只能覆盖未占用槽位，完成后必须不存在 UNCLAIMED。因此

$$\sum_i (end_i - start_i) = L$$

只是必要条件，程序还同时验证 Span 互不重叠，避免“重复计算恰好掩盖遗漏”。

网络层优先调用系统 curl 并设置超时与重试，保留 urllib 回退；密码学与协议解析不依赖网络库或第三方 Bitcoin SDK。

4 真实交易构造与广播

4.1 课程地址

课程专用 Testnet4 地址如下，报告不披露私钥：

- 发送/找零地址: `tb1qw0mgkymm78hd270asxdcvhlucfhjw72t4ajxge;`
- 接收地址: `tb1qkts77kng6xv0rxldrjjfctk5jmf5nx3ltsndlw.`

私钥文件仅标注为测试网络密钥, 并明确禁止发送主网 BTC。父交易和子交易均由程序先本地重算 txid, 再 POST raw hex 到 Esplora 兼容的 Testnet4 `/api/tx`; 仅当服务器返回值与本地 txid 完全相同才写入 `broadcast_txid`。

4.2 父交易: P2WSH(OP_TRUE) 合并

父交易于北京时间 2026-07-20 15:31:26 广播。核心结果见表 2。

表 2: 父交易广播结果

项目	值
txid	4aa330ae4ecf52c82f38d22149b2b7da79f928e1cdad47bb33b553c093a25f7a
wtxid	7ea8089a0b477ef55a2d1ae80bd24f2025ef6020b1207e224177e9ff52f8b9b7
输入/输出	5 个 330 sat P2WSH(OP_TRUE) 输入; 1 个 1000 sat P2WPKH 输出
金额守恒	$5 \times 330 = 1650 = 1000 + 650$ sat
大小	total=263 B, stripped=246 B, weight=1001 WU, vsize=251 vB
手续费率	$650/251 \approx 2.590$ sat/vB
链上状态	节点返回值与本地 txid 一致; 已在高度 144878 确认
浏览器	https://mempool.space/testnet4/tx/4aa330ae4ecf52c82f38d22149b2b7da79f928e1cdad47bb33b553c093a25f7a

对 5 个输入逐一得到

$\text{SHA256}(51) = 4ae81572f06e1b88fd5ced7a1a000945432e83e1551e6f721ee9c00b8cc33260,$

与各 prevout 的 witness program 完全相同; 执行 OP_TRUE 后最终栈均为 [01], 5/5 输入通过 CleanStack。

4.3 子交易: 课程私钥 P2WPKH 签名

子交易于北京时间 2026-07-20 15:32:13 广播。它花费父交易的 1000 sat 输出, 将 400 sat 发到接收地址, 318 sat 找零, 支付 282 sat 手续费。结果见表 3。

表 3: 课程私钥签名交易结果

项目	值
txid	ad00f80a9c85264ebf3ca4299d61cca06efba0487223d4e6caa9d9e58e9ac105
wtxid	56508ced1dc61e19699ce54b23728fbf076b95bb737ead63074926bde6661825
输入	父交易 4aa330ae...25f7a:0, 1000 sat
输出 0	400 sat, 0014b2e1ef5a68d198f19bed1ca49c2ed496d3499a3f
输出 1	318 sat, 001473f68b137bf1eed579fd819b865ffcc26f27794b
金额守恒	1000 = 400 + 318 + 282 sat
大小	total=223 B, stripped=113 B, weight=562 WU, vsize=141 vB
手续费率	282/141 = 2.000 sat/vB
链上状态	节点返回值与本地 txid 一致; 最终采样时仍在 mempool, 未确认
浏览器	https://mempool.space/testnet4/tx/ad00f80a9c85264ebf3ca4299d61cca06efba0487223d4e6caa9d9e58e9ac105

完整 raw transaction 如下; 为适配版心按 64 个十六进制字符换行, 逐行连续拼接即为广播的原始字节流:

```
020000000001017a5fa293c053b533bb47adcde128f979dab7b24921d2382fc852
cf4eae30a34a0000000000fdffffff029001000000000000160014b2e1ef5a68
d198f19bed1ca49c2ed496d3499a3f3e0100000000000016001473f68b137bf1
eed579fd819b865ffcc26f27794b02483045022100b62d9b08f0c3ad4047342a
89926efc9555b41f011ef38825b3f16e1edaca5faf0220522e664de587467051
5a5f6049d92d7db9875d5c05b8deda8a65007539187d6a0121027bcb30196c0f
faee495d2c054e1bf7483bfa470c08dee12cef47d729d5cb50b200000000
```

```

BITCOIN TESTNET4 BYTE AUDIT
$ python chain_analyzer.py --network testnet4 --txid ad00f80a...

[OK] txid      ad00f80a9c85264ebf3ca4299d61cca06efba0487223d4e6caa9d9e58e9ac105
[OK] bytes     223 / 223 accounted
[OK] BIP143    d403cad3a7ad30cf0082c4b8658397c70cbd8dc3fff2ae43539f256793dd93a
[OK] Script    ECDSA=True low-S=True CleanStack=True

[OK] block     0000000000b9d2773b283332f6e1c36da7fdd7b11af12fa7f14866e0beb425ca
[OK] full parse 4,045 bytes, 14 transactions
[OK] Merkle    5853640bbd50baf9df86ae513c514b856ba78e39933d2ea5fda3df2518edd102
[OK] PoW      hash <= 00000000ffff00000000000000000000000000000000000000000000000000000

RESULT: ALL INDEPENDENT CHECKS PASSED

```

图 2：程序根据实际 JSON 输出生成的运行验证汇总图

5 课程签名交易逐字节解析

5.1 字段级完整表

表 4 从偏移 0 连续覆盖到偏移 223；零长度 scriptSig 合法地占据区间 [44, 44)，不消耗字节。prev txid 的 wire bytes 为小端，常见显示值需反转。

表 4：223 字节课程签名交易的字段映射

起点	终点	长度	字段	wire hex / 解释
0	4	4	version	02000000，小端整数 2
4	5	1	marker	00，SegWit marker
5	6	1	flag	01，witness present
6	7	1	input_count	01，1 个输入
7	39	32	prev_txid_le	7a5fa293c053b533bb47adcde128f979dab7b24921d2382fc852cf4eae30a34a
39	43	4	prev_vout	00000000，索引 0
43	44	1	scriptSig length	00

起点	终点	长度	字段	wire hex / 解释
44	44	0	scriptSig	空 (native witness 必须为空)
44	48	4	sequence	fdffffff, 4294967293, 启用 RBF
48	49	1	output_count	02
49	57	8	vout[0].value	9001000000000000, 400 sat
57	58	1	vout[0].script length	16, 22 B
58	80	22	vout[0].scriptPubKey	0014b2e1ef5a68d198f19bed1ca49c2ed496d3499a3f
80	88	8	vout[1].value	3e01000000000000, 318 sat
88	89	1	vout[1].script length	16, 22 B
89	111	22	vout[1].scriptPubKey	001473f68b137bf1eed579fd819b865ffcc26f27794b
111	112	1	witness count	02, 2 项
112	113	1	signature length	48, 72 B
113	185	72	DER signature + type	3045022100b62d9b08f0c3ad4047342a89926efc9555b41f011ef38825b3f16e1edaca5faf0220522e664de5874670515a5f6049d92d7db9875d5c05b8deda8a65007539187d6a01
185	186	1	pubkey length	21, 33 B
186	219	33	compressed pubkey	027bcb30196c0ffaee495d2c054e1bf7483bfa470c08dee12cef47d729d5cb50b2
219	223	4	locktime	00000000, 0

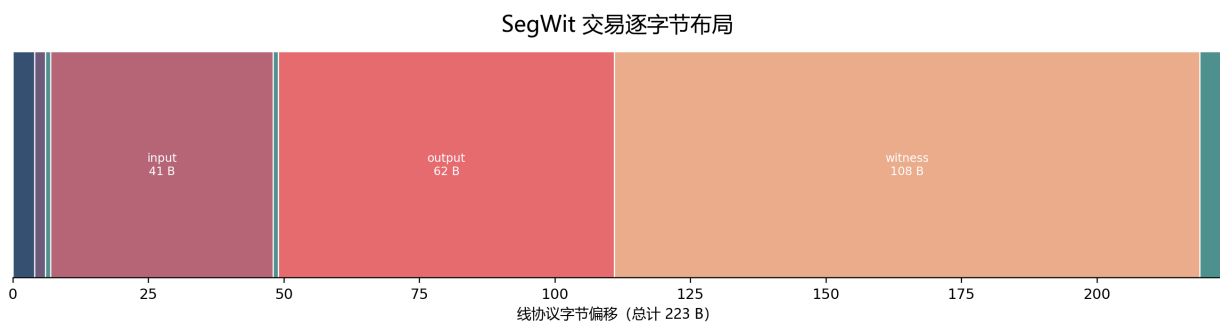


图 3: 课程签名交易各字节区间的可视化

5.2 位级展开示例与完整产物

“down to every bit”按可审计形式实现为：每个 wire byte 均输出偏移、十六进制、恰好 8 位的二进制字符串和字段名。前 16 字节如下：

偏移	hex	bits	字段
0	02	00000010	tx.version
1	00	00000000	tx.version
2	00	00000000	tx.version
3	00	00000000	tx.version
4	00	00000000	tx.marker
5	01	00000001	tx.flag
6	01	00000001	tx.input_count
7	7a	01111010	vin[0].prev_txid_le
8	5f	01011111	vin[0].prev_txid_le
9	a2	10100010	vin[0].prev_txid_le
10	93	10010011	vin[0].prev_txid_le
11	c0	11000000	vin[0].prev_txid_le
12	53	01010011	vin[0].prev_txid_le
13	b5	10110101	vin[0].prev_txid_le
14	33	00110011	vin[0].prev_txid_le
15	bb	10111011	vin[0].prev_txid_le

完整 223 行位图位于 `output/testnet4_signed_analysis/transaction_bits.txt`；完整区块 4045 行位图位于 `output/testnet4_signed_analysis/block_bits.txt`。这两个文件是报告电子附件，行数分别为 $223 + 1$ 与 $4045 + 1$ （第一行为表头）。

6 BIP143 与 Script 严格验证

6.1 签名中间值

本输入被花费金额为 1000 sat，scriptCode 为

76a91473f68b137bf1eed579fd819b865ffcc26f27794b88ac.

程序给出的三个聚合哈希为：

```
hashPrevouts = d6f3e525d7a564b2ba16490b4806fb12c5767e126f4cacda653d5db02be1d48e
hashSequence = caf35e5224de16efa3ccaf41070f6e7b9432b6f79551e629fca9d1c03b43bc52
```

```
hashOutputs = d23fabf0910579f82d571b5d5e580265b9b454353b77bb590de4e8700df80247
```

完整原像如下；同样按 64 个十六进制字符换行，计算时不包含换行符：

```
02000000d6f3e525d7a564b2ba16490b4806fb12c5767e126f4cacda653d5db0
2be1d48ecaf35e5224de16efa3ccaf41070f6e7b9432b6f79551e629fca9d1c
03b43bc527a5fa293c053b533bb47adcde128f979dab7b24921d2382fc852cf4e
ae30a34a000000001976a91473f68b137bf1eed579fd819b865ffcc26f27794b
88ace80300000000000000fdfffffd23fabf0910579f82d571b5d5e580265b9b4
54353b77bb590de4e8700df802470000000001000000
```

故

$$z = \text{SHA256d}(\text{preimage})$$
$$\text{d403cadc3a7ad30cf0082c4b8658397c70cbd8dc3fff2ae43539f256793dd93a.}$$

DER 解码得到 $r = \text{b62d9b08f0c3ad4047342a89926efc9555b41f011ef38825b3f16e1edaca5faf}$

$s = 522e664de5874670515a5f6049d92d7db9875d5c05b8deda8a65007539187d6a$. 程序验证 $1 \leq r, s < n$ 、 $s \leq n/2$ ，并独立完成 $s^{-1}(zG + rQ)$ 点运算，所得横坐标模 n 等于 r 。

6.2 P2WPKH 栈执行

表 5: 课程签名输入的 Script 执行轨迹

步骤	动作与检查	栈深度/结论
Witness 初始化	压入 DER 签名 +01 与 33 B 压缩公钥	2
OP_DUP	复制栈顶公钥	3
OP_HASH160	对复制公钥计算 RIPEMD160(SHA256(pubkey))	3
PUSH20	从 prevout witness program 压入 73f68b...7794b	4
OP_EQUALVERIFY	两个 20 B 哈希相等并弹出	2
OP_CHECKSIG	用 BIP143 摘要与公钥验证 ECDSA	[01]
CleanStack	最终恰有一个真值	通过

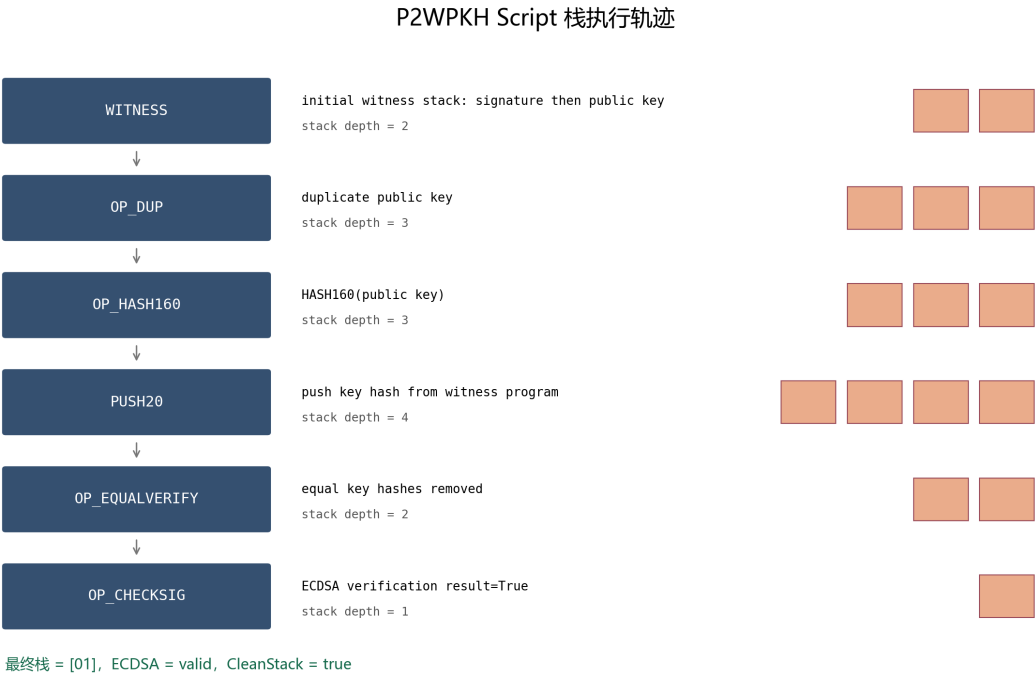


图 4: P2WPKH Script 栈逐步执行结果

7 完整区块解析与 PoW 验证

父交易随后在 Testnet4 高度 144878 获得确认，所在区块为 0000000000b9d2773b283332f6e1c36da7fdd7b11af12fa7f14866e0beb425ca。因此本节解析该真实包含父交易的完整区块；父交易位于交易索引 13（从 0 起，即区块内第 14 笔）。逐字节交易对象仍选用具有 BIP143/ECDSA 细节的子交易。最终采样时子交易尚未确认，故不虚构其与该区块之间的包含关系。


```
-----  
Ran 6 tests in 0.366s  
OK
```

覆盖要点包括：标准 secp256k1 基点压缩公钥、签名消息篡改必须失败、low-S、DER 往返、Bech32 往返、非最短 CompactSize 必须拒绝、真实交易反序列化/重序列化、P2WPKH Script、P2WSH(OP_TRUE)、完整区块 Merkle/PoW 与字节行数。

8.2 复现实验命令

在作业一跟目录中运行：

```
python -m unittest discover -s code -p "test_*.py" -v  
  
python code/wallet_tool.py new --network testnet4 \  
--wallet data/testnet4_sender_wallet.json  
  
python code/claim_op_true_utxos.py \  
--wallet data/testnet4_sender_wallet.json \  
--inputs 5 --output-value 1000 --broadcast  
  
python code/wallet_tool.py send \  
--wallet data/testnet4_sender_wallet.json \  
--destination tb1qkts77kng6xvOrxldrjffctk5jmf5nx3ltsndlW \  
--amount 400 --fee-rate 2 --allow-unconfirmed --broadcast \  
--output output/testnet4_signed_broadcast.json  
  
python code/chain_analyzer.py --network testnet4 \  
--txid ad00f80a9c85264ebf3ca4299d61cca06efba0487223d4e6caa9d9e58e9ac105 \  
--block-hash 0000000000b9d2773b283332f6e1c36da7fdd7b11af12fa7f14866e0beb425ca \  
--output output/testnet4_signed_analysis  
  
python code/visualize_results.py
```

8.3 威胁模型与局限

边界说明

本实现是为了展示每个字节和每一步数学计算的课程代码，不是生产钱包。纯 Python 大整数点运算未承诺恒定时间；测试私钥只可用于无价值测试网络。程序验证本题涉及的 P2WPKH 与 P2WSH(OP_TRUE)，并不宣称实现完整 Bitcoin Script 指令集或完整节点共识验证。REST API 只负责传输 raw bytes；真实性由本地 txid、block hash、Merkle 与 PoW 重算交叉检查。

广播时父子交易均被 Testnet4 节点 mempool 接受；最终采样时父交易已经在高度 144878 确认，子交易仍未确认。mempool 接受证明交易已发送并通过节点策略验证，但不等同于最终确认；对子交易仍只陈述节点接受这一事实。报告同时保留 raw transaction、节点响应与本地重算结果，因此即使后续链上状态变化，实验数据仍可独立复核。

9 结论

本作业完成了原 Project 的四个核心动作：真实测试网发送、交易逐位/逐字节解析、完整区块解析以及 Script 逐步计算。父交易从 P2WSH witness program 出发验证公开 OP_TRUE 授权并已在高度 144878 确认；子交易从 BIP143 原像出发完成 RFC6979/low-S ECDSA 签名并由节点接受。独立解析器对 223 B 子交易和包含父交易的 4045 B 区块做到无遗漏、无重叠映射，重算 txid、wtxid、Merkle 根、目标值和 PoW；Script 最终均为 CleanStack TRUE。默认 Signet 的已确认样本进一步验证实现的泛化性。结果文件与完整源代码共同构成可复现、可审计的交付。

A 完整逐字节结果文件索引

相对路径	内容
output/testnet4_signed_analysis/transaction_bits.txt	课程签名交易 223 个字节的 8 位展开
output/testnet4_signed_analysis/transaction_fields.csv	字段起止偏移、wire hex、解释值
output/testnet4_signed_analysis/script_audit.json	BIP143 中间值、DER、ECDSA、Script 栈
output/testnet4_signed_analysis/block_bits.txt	完整区块 4045 个字节的 8 位展开
output/testnet4_signed_analysis/block_fields.csv	完整区块字段映射
output/testnet4_signed_analysis/block_transaction_index.csv	14 笔交易的 txid/wtxid/size/weight/v-size
output/testnet4_funding_analysis/script_audit.json	5 个 P2WSH(OP_TRUE) 输入的执行轨迹
output/sample_confirmed/	默认 Signet 已确认交易和 74123 B 区块回归结果

B 核心源代码

以下代码均为本作业实际运行版本。

B.1 bitcoin_codec.py

```

1  """Minimal Bitcoin wire-format, secp256k1, and Script utilities.
2
3  This module intentionally uses only the Python standard library so that every
4  byte and every cryptographic calculation remains inspectable for coursework.
5  It is not intended to replace a production Bitcoin wallet.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass, field
11 import hashlib
12 import hmac
13 import math

```

```

14 import struct
15 from typing import Any
16
17
18 P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F
19 N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAEDCE6AF48A03BBFD25E8CD0364141
20 GX = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
21 GY = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
22 G = (GX, GY)
23
24
25 def sha256(data: bytes) -> bytes:
26     return hashlib.sha256(data).digest()
27
28
29 def sha256d(data: bytes) -> bytes:
30     return sha256(sha256(data))
31
32
33 def hash160(data: bytes) -> bytes:
34     return hashlib.new("ripemd160", sha256(data)).digest()
35
36
37 def inv_mod(value: int, modulus: int) -> int:
38     return pow(value % modulus, -1, modulus)
39
40
41 def point_add(a: tuple[int, int] | None, b: tuple[int, int] | None) -> tuple[int, int] | None:
42     if a is None:
43         return b
44     if b is None:
45         return a
46     x1, y1 = a
47     x2, y2 = b
48     if x1 == x2 and (y1 + y2) % P == 0:
49         return None
50     if a == b:
51         slope = (3 * x1 * x1) * inv_mod(2 * y1, P) % P
52     else:
53         slope = (y2 - y1) * inv_mod(x2 - x1, P) % P
54     x3 = (slope * slope - x1 - x2) % P
55     y3 = (slope * (x1 - x3) - y1) % P
56     return x3, y3
57
58
59 def scalar_mult(k: int, point: tuple[int, int] | None = G) -> tuple[int, int] | None:
60     if point is None or k % N == 0:
61         return None
62     if k < 0:
63         return scalar_mult(-k, (point[0], (-point[1]) % P))
64     result = None
65     addend = point
66     while k:
67         if k & 1:
68             result = point_add(result, addend)
69             addend = point_add(addend, addend)
70         k >>= 1
71     return result
72
73

```

```

74 def encode_pubkey(point: tuple[int, int], compressed: bool = True) -> bytes:
75     x, y = point
76     if compressed:
77         return bytes([2 + (y & 1)]) + x.to_bytes(32, "big")
78     return b"\x04" + x.to_bytes(32, "big") + y.to_bytes(32, "big")
79
80
81 def decode_pubkey(data: bytes) -> tuple[int, int]:
82     if len(data) == 33 and data[0] in (2, 3):
83         x = int.from_bytes(data[1:], "big")
84         if x >= P:
85             raise ValueError("compressed public key x-coordinate is out of range")
86         y = pow((pow(x, 3, P) + 7) % P, (P + 1) // 4, P)
87         if (y & 1) != (data[0] & 1):
88             y = P - y
89     elif len(data) == 65 and data[0] == 4:
90         x = int.from_bytes(data[1:33], "big")
91         y = int.from_bytes(data[33:], "big")
92     else:
93         raise ValueError("unsupported public-key encoding")
94     if x >= P or y >= P or (y * y - x * x * x - 7) % P:
95         raise ValueError("point is not on secp256k1")
96     return x, y
97
98
99 def rfc6979_nonce(secret: int, digest: bytes) -> int:
100     """Generate the RFC6979 HMAC-SHA256 nonce for secp256k1."""
101     x = secret.to_bytes(32, "big")
102     h1 = (int.from_bytes(digest, "big") % N).to_bytes(32, "big")
103     v = b"\x01" * 32
104     k = b"\x00" * 32
105     k = hmac.new(k, v + b"\x00" + x + h1, hashlib.sha256).digest()
106     v = hmac.new(k, v, hashlib.sha256).digest()
107     k = hmac.new(k, v + b"\x01" + x + h1, hashlib.sha256).digest()
108     v = hmac.new(k, v, hashlib.sha256).digest()
109     while True:
110         v = hmac.new(k, v, hashlib.sha256).digest()
111         candidate = int.from_bytes(v, "big")
112         if 1 <= candidate < N:
113             return candidate
114     k = hmac.new(k, v + b"\x00", hashlib.sha256).digest()
115     v = hmac.new(k, v, hashlib.sha256).digest()
116
117
118 def ecdsa_sign(secret: int, digest: bytes) -> tuple[int, int]:
119     if not 1 <= secret < N or len(digest) != 32:
120         raise ValueError("invalid ECDSA signing input")
121     z = int.from_bytes(digest, "big")
122     nonce = rfc6979_nonce(secret, digest)
123     point = scalar_mult(nonce)
124     assert point is not None
125     r = point[0] % N
126     if r == 0:
127         raise RuntimeError("RFC6979 produced an unusable nonce")
128     s = inv_mod(nonce, N) * (z + r * secret) % N
129     if s == 0:
130         raise RuntimeError("RFC6979 produced an unusable signature")
131     if s > N // 2:
132         s = N - s
133     return r, s

```

```

134
135
136 def ecdsa_verify(pubkey: bytes, digest: bytes, r: int, s: int) -> bool:
137     if len(digest) != 32 or not (1 <= r < N and 1 <= s < N):
138         return False
139     try:
140         public_point = decode_pubkey(pubkey)
141     except ValueError:
142         return False
143     z = int.from_bytes(digest, "big")
144     w = inv_mod(s, N)
145     result = point_add(scalar_mult(z * w % N), scalar_mult(r * w % N, public_point))
146     return result is not None and result[0] % N == r
147
148
149 def der_encode_signature(r: int, s: int) -> bytes:
150     def der_int(value: int) -> bytes:
151         raw = value.to_bytes((value.bit_length() + 7) // 8 or 1, "big")
152         if raw[0] & 0x80:
153             raw = b"\x00" + raw
154         return b"\x02" + bytes([len(raw)]) + raw
155
156     body = der_int(r) + der_int(s)
157     return b"\x30" + bytes([len(body)]) + body
158
159
160 def der_decode_signature(data: bytes) -> tuple[int, int]:
161     if len(data) < 8 or data[0] != 0x30 or data[1] != len(data) - 2:
162         raise ValueError("invalid DER signature sequence")
163     if data[2] != 0x02:
164         raise ValueError("invalid DER r tag")
165     r_len = data[3]
166     r_start, r_end = 4, 4 + r_len
167     if r_end + 2 > len(data) or data[r_end] != 0x02:
168         raise ValueError("invalid DER s tag")
169     s_len = data[r_end + 1]
170     s_start, s_end = r_end + 2, r_end + 2 + s_len
171     if s_end != len(data) or not r_len or not s_len:
172         raise ValueError("invalid DER integer length")
173     r_raw, s_raw = data[r_start:r_end], data[s_start:s_end]
174     for raw in (r_raw, s_raw):
175         if raw[0] & 0x80 or (len(raw) > 1 and raw[0] == 0 and not raw[1] & 0x80):
176             raise ValueError("non-minimal or negative DER integer")
177     return int.from_bytes(r_raw, "big"), int.from_bytes(s_raw, "big")
178
179
180 BASE58_ALPHABET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz"
181
182
183 def base58check(payload: bytes) -> str:
184     raw = payload + sha256d(payload)[:4]
185     number = int.from_bytes(raw, "big")
186     encoded = ""
187     while number:
188         number, remainder = divmod(number, 58)
189         encoded = BASE58_ALPHABET[remainder] + encoded
190     return "1" * (len(raw) - len(raw.lstrip(b"\x00"))) + encoded
191
192
193 BECH32_CHARSET = "qpzry9x8gf2tvdw0s3jn54khce6mua7l"

```

```

194
195
196 def _bech32_polymod(values: list[int]) -> int:
197     generators = (0x3B6A57B2, 0x26508E6D, 0x1EA119FA, 0x3D4233DD, 0x2A1462B3)
198     chk = 1
199     for value in values:
200         top = chk >> 25
201         chk = ((chk & 0xFFFFFFFF) << 5) ^ value
202         for i, generator in enumerate(generators):
203             if (top >> i) & 1:
204                 chk ^= generator
205     return chk
206
207
208 def _bech32_hrp_expand(hrp: str) -> list[int]:
209     return [ord(x) >> 5 for x in hrp] + [0] + [ord(x) & 31 for x in hrp]
210
211
212 def convert_bits(data: bytes, from_bits: int, to_bits: int, pad: bool = True) -> list[int]:
213     accumulator = 0
214     bits = 0
215     result: list[int] = []
216     max_value = (1 << to_bits) - 1
217     for value in data:
218         if value < 0 or value >> from_bits:
219             raise ValueError("invalid value for bit conversion")
220         accumulator = (accumulator << from_bits) | value
221         bits += from_bits
222         while bits >= to_bits:
223             bits -= to_bits
224             result.append((accumulator >> bits) & max_value)
225     if pad:
226         if bits:
227             result.append((accumulator << (to_bits - bits)) & max_value)
228     elif bits >= from_bits or ((accumulator << (to_bits - bits)) & max_value):
229         raise ValueError("invalid non-zero padding")
230     return result
231
232
233 def segwit_address(program: bytes, hrp: str = "tb", version: int = 0) -> str:
234     if version != 0 or len(program) not in (20, 32):
235         raise ValueError("this coursework wallet supports only witness v0")
236     data = [version] + convert_bits(program, 8, 5)
237     values = _bech32_hrp_expand(hrp) + data
238     polymod = _bech32_polymod(values + [0] * 6) ^ 1
239     checksum = [(polymod >> (5 * (5 - i))) & 31 for i in range(6)]
240     return hrp + "1" + "".join(BECH32_CHARSET[x] for x in data + checksum)
241
242
243 def decode_segwit_address(address: str) -> tuple[str, int, bytes]:
244     if address.lower() != address and address.upper() != address:
245         raise ValueError("mixed-case Bech32 address")
246     address = address.lower()
247     separator = address.rfind("1")
248     if separator < 1 or separator + 7 > len(address):
249         raise ValueError("invalid Bech32 separator position")
250     hrp = address[:separator]
251     try:
252         data = [BECH32_CHARSET.index(char) for char in address[separator + 1 :]]
253     except ValueError as exc:

```

```

254         raise ValueError("invalid Bech32 character") from exc
255     if _bech32_polymod(_bech32_hrp_expand(hrp) + data) != 1:
256         raise ValueError("invalid Bech32 checksum")
257     version = data[0]
258     converted = convert_bits(bytes(data[1:-6]), 5, 8, pad=False)
259     program = bytes(converted)
260     if version != 0 or len(program) not in (20, 32):
261         raise ValueError("unsupported witness version or program length")
262     return hrp, version, program
263
264
265 def encode_varint(value: int) -> bytes:
266     if value < 0:
267         raise ValueError("CompactSize cannot be negative")
268     if value < 0xFD:
269         return bytes([value])
270     if value <= 0xFFFF:
271         return b"\xfd" + struct.pack("<H", value)
272     if value <= 0xFFFFFFFF:
273         return b"\xfe" + struct.pack("<I", value)
274     if value <= 0xFFFFFFFFFFFFFFFF:
275         return b"\xff" + struct.pack("<Q", value)
276     raise ValueError("CompactSize is too large")
277
278
279 @dataclass
280 class Span:
281     start: int
282     end: int
283     name: str
284     value: str
285
286
287 class ByteReader:
288     def __init__(self, data: bytes, offset: int = 0):
289         self.data = data
290         self.pos = offset
291         self.spans: list[Span] = []
292
293     def take(self, length: int, name: str, value: str | None = None) -> bytes:
294         if length < 0 or self.pos + length > len(self.data):
295             raise ValueError(f"truncated data while reading {name} at offset {self.pos}")
296         start = self.pos
297         raw = self.data[start : start + length]
298         self.pos += length
299         self.spans.append(Span(start, self.pos, name, value if value is not None else raw.hex()))
300         return raw
301
302     def uint32(self, name: str) -> int:
303         start = self.pos
304         raw = self.take(4, name)
305         value = struct.unpack("<I", raw)[0]
306         self.spans[-1] = Span(start, self.pos, name, str(value))
307         return value
308
309     def uint64(self, name: str) -> int:
310         start = self.pos
311         raw = self.take(8, name)
312         value = struct.unpack("<Q", raw)[0]
313         self.spans[-1] = Span(start, self.pos, name, str(value))

```



```

314         return value
315
316     def varint(self, name: str) -> int:
317         start = self.pos
318         first = self.take(1, name)[0]
319         if first < 0xFD:
320             value = first
321         else:
322             width = {0xFD: 2, 0xFE: 4, 0xFF: 8}[first]
323             tail = self.take(width, name + ".payload")
324             value = int.from_bytes(tail, "little")
325             minimum = {2: 0xFD, 4: 0x10000, 8: 0x100000000}[width]
326             if value < minimum:
327                 raise ValueError(f"non-minimal CompactSize at offset {start}")
328             self.spans.pop()
329             self.spans[-1] = Span(start, self.pos, name, str(value))
330         return value
331
332
333 @dataclass
334 class TxInput:
335     prev_txid: str
336     prev_vout: int
337     script_sig: bytes
338     sequence: int
339     witness: list[bytes] = field(default_factory=list)
340
341     def serialize(self) -> bytes:
342         return (
343             bytes.fromhex(self.prev_txid)[-1:]
344             + struct.pack("<I", self.prev_vout)
345             + encode_varint(len(self.script_sig))
346             + self.script_sig
347             + struct.pack("<I", self.sequence)
348         )
349
350
351 @dataclass
352 class TxOutput:
353     value: int
354     script_pubkey: bytes
355
356     def serialize(self) -> bytes:
357         return struct.pack("<Q", self.value) + encode_varint(len(self.script_pubkey)) + self.script_pubkey
358
359
360 @dataclass
361 class Transaction:
362     version: int
363     inputs: list[TxInput]
364     outputs: list[TxOutput]
365     locktime: int
366     segwit: bool = False
367     spans: list[Span] = field(default_factory=list)
368     raw: bytes = b""
369
370     def serialize(self, include_witness: bool = True) -> bytes:
371         use_witness = include_witness and self.segwit
372         result = struct.pack("<I", self.version)
373         if use_witness:

```

```

374         result += b"\x00\x01"
375     result += encode_varint(len(self.inputs))
376     result += b"".join(item.serialize() for item in self.inputs)
377     result += encode_varint(len(self.outputs))
378     result += b"".join(item.serialize() for item in self.outputs)
379     if use_witness:
380         for txin in self.inputs:
381             result += encode_varint(len(txin.witness))
382             for item in txin.witness:
383                 result += encode_varint(len(item)) + item
384     return result + struct.pack("<I", self.locktime)
385
386 @property
387 def txid(self) -> str:
388     return sha256d(self.serialize(include_witness=False))[:-1].hex()
389
390 @property
391 def wtxid(self) -> str:
392     return sha256d(self.serialize(include_witness=True))[:-1].hex()
393
394 @property
395 def stripped_size(self) -> int:
396     return len(self.serialize(include_witness=False))
397
398 @property
399 def total_size(self) -> int:
400     return len(self.serialize(include_witness=True))
401
402 @property
403 def weight(self) -> int:
404     return self.stripped_size * 4 + (self.total_size - self.stripped_size)
405
406 @property
407 def vsize(self) -> int:
408     return math.ceil(self.weight / 4)
409
410
411 def parse_transaction(data: bytes, offset: int = 0, prefix: str = "tx") -> tuple[Transaction, int]:
412     reader = ByteReader(data, offset)
413     start = offset
414     version = reader.uint32(prefix + ".version")
415     segwit = data[reader.pos : reader.pos + 2] == b"\x00\x01"
416     if segwit:
417         reader.take(1, prefix + ".marker", "0 (SegWit marker)")
418         reader.take(1, prefix + ".flag", "1 (witness present)")
419     input_count = reader.varint(prefix + ".input_count")
420     inputs: list[TxInput] = []
421     for index in range(input_count):
422         item_prefix = f"{prefix}.vin[{index}]"
423         prev_hash_raw = reader.take(32, item_prefix + ".prev_txid_le")
424         prev_vout = reader.uint32(item_prefix + ".prev_vout")
425         script_length = reader.varint(item_prefix + ".script_sig_length")
426         script_sig = reader.take(script_length, item_prefix + ".script_sig")
427         sequence = reader.uint32(item_prefix + ".sequence")
428         inputs.append(TxInput(prev_hash_raw[:-1].hex(), prev_vout, script_sig, sequence))
429     output_count = reader.varint(prefix + ".output_count")
430     outputs: list[TxOutput] = []
431     for index in range(output_count):
432         item_prefix = f"{prefix}.vout[{index}]"
433         value = reader.uint64(item_prefix + ".value_satoshi")

```

```

434     script_length = reader.varint(item_prefix + ".script_pubkey_length")
435     script_pubkey = reader.take(script_length, item_prefix + ".script_pubkey")
436     outputs.append(TxOutput(value, script_pubkey))
437     if segwit:
438         for input_index, txin in enumerate(inputs):
439             item_count = reader.varint(f"{prefix}.vin[{input_index}].witness_count")
440             for item_index in range(item_count):
441                 length = reader.varint(f"{prefix}.vin[{input_index}].witness[{item_index}].length")
442                 txin.witness.append(
443                     reader.take(length, f"{prefix}.vin[{input_index}].witness[{item_index}].data")
444                 )
445     locktime = reader.uint32(prefix + ".locktime")
446     tx = Transaction(version, inputs, outputs, locktime, segwit, reader.spans, data[start : reader.pos])
447     if tx.raw != tx.serialize(include_witness=True):
448         raise AssertionError("transaction reserialization differs from wire bytes")
449     return tx, reader.pos
450
451
452 @dataclass
453 class Block:
454     version: int
455     previous_hash: str
456     merkle_root: str
457     timestamp: int
458     bits: int
459     nonce: int
460     transactions: list[Transaction]
461     spans: list[Span]
462     raw: bytes
463
464     @property
465     def header(self) -> bytes:
466         return self.raw[:80]
467
468     @property
469     def block_hash(self) -> str:
470         return sha256d(self.header)[:1].hex()
471
472     @property
473     def computed_merkle_root(self) -> str:
474         hashes = [bytes.fromhex(tx.txid)[:1] for tx in self.transactions]
475         if not hashes:
476             raise ValueError("block has no transactions")
477         while len(hashes) > 1:
478             if len(hashes) & 1:
479                 hashes.append(hashes[-1])
480             hashes = [sha256d(hashes[i] + hashes[i + 1]) for i in range(0, len(hashes), 2)]
481         return hashes[0][:1].hex()
482
483     @property
484     def target(self) -> int:
485         exponent = self.bits >> 24
486         coefficient = self.bits & 0x007FFFFF
487         return coefficient * (1 << (8 * (exponent - 3)))
488
489
490 def parse_block(data: bytes) -> Block:
491     if len(data) < 81:
492         raise ValueError("truncated block")
493     reader = ByteReader(data)

```

```

494     version = reader.uint32("block.header.version")
495     previous_hash = reader.take(32, "block.header.previous_hash_le")[:-1].hex()
496     merkle_root = reader.take(32, "block.header.merkle_root_le")[:-1].hex()
497     timestamp = reader.uint32("block.header.timestamp")
498     bits = reader.uint32("block.header.bits")
499     nonce = reader.uint32("block.header.nonce")
500     tx_count = reader.varint("block.transaction_count")
501     transactions: list[Transaction] = []
502     all_spans = list(reader.spans)
503     position = reader.pos
504     for index in range(tx_count):
505         tx, position = parse_transaction(data, position, f"block.tx[{index}]")
506         transactions.append(tx)
507         all_spans.extend(tx.spans)
508     if position != len(data):
509         raise ValueError(f"{len(data) - position} trailing bytes after block")
510     return Block(version, previous_hash, merkle_root, timestamp, bits, nonce, transactions, all_spans, data)
511
512
513 def p2wpkh_script(program: bytes) -> bytes:
514     if len(program) != 20:
515         raise ValueError("P2WPKH program must be 20 bytes")
516     return b"\x00\x14" + program
517
518
519 def bip143_sighash_details(
520     tx: Transaction, input_index: int, script_code: bytes, value: int
521 ) -> dict[str, bytes]:
522     if not 0 <= input_index < len(tx.inputs):
523         raise IndexError("input index out of range")
524     hash_prevouts = sha256d(
525         b"".join(bytes.fromhex(item.prev_txid)[:-1] + struct.pack("<I", item.prev_vout) for item in tx.inputs)
526     )
527     hash_sequence = sha256d(b"".join(struct.pack("<I", item.sequence) for item in tx.inputs))
528     hash_outputs = sha256d(b"".join(item.serialize() for item in tx.outputs))
529     txin = tx.inputs[input_index]
530     preimage = (
531         struct.pack("<I", tx.version)
532         + hash_prevouts
533         + hash_sequence
534         + bytes.fromhex(txin.prev_txid)[:-1]
535         + struct.pack("<I", txin.prev_vout)
536         + encode_varint(len(script_code))
537         + script_code
538         + struct.pack("<Q", value)
539         + struct.pack("<I", txin.sequence)
540         + hash_outputs
541         + struct.pack("<I", tx.locktime)
542         + struct.pack("<I", 1)
543     )
544     return {
545         "hash_prevouts": hash_prevouts,
546         "hash_sequence": hash_sequence,
547         "hash_outputs": hash_outputs,
548         "preimage": preimage,
549         "digest": sha256d(preimage),
550     }
551
552
553 def bip143_sighash_all(tx: Transaction, input_index: int, script_code: bytes, value: int) -> bytes:

```

```

554     return bip143_sighash_details(tx, input_index, script_code, value)["digest"]
555
556
557 def verify_p2wpkh_input(
558     tx: Transaction, input_index: int, prevout_script: bytes, prevout_value: int
559 ) -> dict[str, Any]:
560     """Execute the P2WPKH-equivalent P2PKH script and return an audit trace."""
561     if len(prevout_script) != 22 or prevout_script[2] != b"\x00\x14":
562         raise ValueError("previous output is not P2WPKH")
563     witness = tx.inputs[input_index].witness
564     if len(witness) != 2:
565         raise ValueError("P2WPKH witness must contain signature and public key")
566     signature_with_type, public_key = witness
567     if not signature_with_type or signature_with_type[-1] != 1:
568         raise ValueError("only SIGHASH_ALL is supported")
569     r, s = der_decode_signature(signature_with_type[:-1])
570     expected_hash = prevout_script[2:]
571     script_code = b"\x76\xa9\x14" + expected_hash + b"\x88\xac"
572     sighash = bip143_sighash_details(tx, input_index, script_code, prevout_value)
573     digest = sighash["digest"]
574     stack: list[bytes] = [signature_with_type, public_key]
575     trace: list[dict[str, Any]] = []
576
577 def snapshot(operation: str, detail: str) -> None:
578     trace.append({"operation": operation, "detail": detail, "stack": [item.hex() for item in stack]})
579
580 snapshot("WITNESS", "initial witness stack: signature then public key")
581 stack.append(stack[-1])
582 snapshot("OP_DUP", "duplicate public key")
583 stack.append(hash160(stack.pop()))
584 snapshot("OP_HASH160", "HASH160(public key)")
585 stack.append(expected_hash)
586 snapshot("PUSH20", "push key hash from witness program")
587 right, left = stack.pop(), stack.pop()
588 if left != right:
589     raise ValueError("OP_EQUALVERIFY failed: public-key hash mismatch")
590 snapshot("OP_EQUALVERIFY", "equal key hashes removed")
591 public_key_from_stack = stack.pop()
592 signature_from_stack = stack.pop()
593 verified = ecdsa_verify(public_key_from_stack, digest, r, s)
594 stack.append(b"\x01" if verified else b"")
595 snapshot("OP_CHECKSIG", f"ECDSA verification result={verified}")
596 cleanstack = len(stack) == 1 and stack[-1] not in (b"", b"\x00")
597 return {
598     "script_type": "P2WPKH",
599     "script_code": script_code.hex(),
600     "sighash_digest": digest.hex(),
601     "hash_prevouts": sighash["hash_prevouts"].hex(),
602     "hash_sequence": sighash["hash_sequence"].hex(),
603     "hash_outputs": sighash["hash_outputs"].hex(),
604     "sighash_preimage": sighash["preimage"].hex(),
605     "r": hex(r),
606     "s": hex(s),
607     "low_s": s <= N // 2,
608     "pubkey_hash_matches": hash160(public_key) == expected_hash,
609     "ecdsa_valid": verified,
610     "cleanstack_success": cleanstack,
611     "trace": trace,
612 }
613

```

```

614
615 def verify_p2wsh_op_true_input(tx: Transaction, input_index: int, prevout_script: bytes) -> dict[str, Any]:
616     """Audit the explicit P2WSH(OP_TRUE) authorization used for test funding."""
617     if len(prevout_script) != 34 or prevout_script[2] != b"\x00\x20":
618         raise ValueError("previous output is not P2WSH")
619     witness = tx.inputs[input_index].witness
620     if witness != [b"\x51"]:
621         raise ValueError("this audit expects a single OP_TRUE witnessScript")
622     witness_script = witness[-1]
623     expected_program = prevout_script[2:]
624     actual_program = sha256(witness_script)
625     program_matches = hmac.compare_digest(actual_program, expected_program)
626     stack: list[bytes] = []
627     trace = [
628         {
629             "operation": "WITNESS_SCRIPT",
630             "detail": "SHA256(witnessScript) equals the 32-byte witness program",
631             "stack": [],
632         }
633     ]
634     if not program_matches:
635         raise ValueError("P2WSH witnessScript hash mismatch")
636     stack.append(b"\x01")
637     trace.append({"operation": "OP_TRUE", "detail": "push script number 1", "stack": ["01"]})
638     cleanstack = len(stack) == 1 and stack[-1] == b"\x01"
639     return {
640         "script_type": "P2WSH(OP_TRUE)",
641         "witness_script": witness_script.hex(),
642         "witness_program_expected": expected_program.hex(),
643         "witness_program_computed": actual_program.hex(),
644         "witness_program_matches": program_matches,
645         "cleanstack_success": cleanstack,
646         "trace": trace,
647     }

```

B.2 wallet_tool.py

```

1  """Create and broadcast a transparent, test-network-only P2WPKH transaction."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import os
8  from pathlib import Path
9  import shutil
10 import subprocess
11 import time
12 import urllib.error
13 import urllib.request
14
15 from bitcoin_codec import (
16     N,
17     Transaction,
18     TxInput,
19     TxOutput,
20     base58check,

```

```

21     bip143_sighash_all,
22     der_encode_signature,
23     ecdsa_sign,
24     encode_pubkey,
25     hash160,
26     p2wpkh_script,
27     scalar_mult,
28     segwit_address,
29 )
30
31
32 API_ROOTS = {
33     "mutinynet": "https://mutinynet.com/api",
34     "signet": "https://mempool.space/signet/api",
35     "testnet": "https://mempool.space/testnet/api",
36     "testnet4": "https://mempool.space/testnet4/api",
37 }
38
39
40 def api_request(url: str, data: bytes | None = None) -> bytes:
41     curl = shutil.which("curl.exe") or shutil.which("curl")
42     if curl:
43         command = [curl, "-sS", "-L", "--retry", "3", "--max-time", "60"]
44         if data is not None:
45             command += ["-H", "Content-Type: text/plain", "--data-binary", "@-"]
46         command.append(url)
47         result = subprocess.run(command, input=data, capture_output=True, check=False)
48         if result.returncode == 0:
49             return result.stdout
50         raise RuntimeError(f"curl failed for {url}: {result.stderr.decode(errors='replace').strip()}")
51     last_error: Exception | None = None
52     for attempt in range(1, 4):
53         request = urllib.request.Request(url, data=data, headers={"User-Agent": "coursework-byte-auditor/1.0"})
54         if data is not None:
55             request.add_header("Content-Type", "text/plain")
56         try:
57             with urllib.request.urlopen(request, timeout=30) as response:
58                 return response.read()
59         except urllib.error.HTTPError as exc:
60             body = exc.read().decode("utf-8", errors="replace")
61             raise RuntimeError(f"HTTP {exc.code} from {url}: {body}") from exc
62         except (urllib.error.URLError, TimeoutError) as exc:
63             last_error = exc
64             if attempt < 3:
65                 time.sleep(attempt)
66     raise RuntimeError(f"network request failed after 3 attempts: {url}: {last_error}")
67
68
69 def create_wallet(path: Path, network: str) -> dict[str, object]:
70     if path.exists():
71         raise FileExistsError(f"refusing to overwrite existing wallet: {path}")
72     secret = int.from_bytes(os.urandom(32), "big")
73     while not 1 <= secret < N:
74         secret = int.from_bytes(os.urandom(32), "big")
75     point = scalar_mult(secret)
76     assert point is not None
77     pubkey = encode_pubkey(point)
78     program = hash160(pubkey)
79     wallet = {
80         "warning": "TEST NETWORK KEY ONLY - NEVER SEND MAINNET BITCOIN TO THIS KEY",

```

```

81     "network": network,
82     "private_key_hex": secret.to_bytes(32, "big").hex(),
83     "wif_testnet_compressed": base58check(b"\xef" + secret.to_bytes(32, "big") + b"\x01"),
84     "public_key_compressed": pubkey.hex(),
85     "p2wpkh_program": program.hex(),
86     "address": segwit_address(program),
87     "created_unix": int(time.time()),
88 }
89 path.parent.mkdir(parents=True, exist_ok=True)
90 path.write_text(json.dumps(wallet, ensure_ascii=False, indent=2), encoding="utf-8")
91 return wallet
92
93
94 def load_wallet(path: Path) -> dict[str, object]:
95     wallet = json.loads(path.read_text(encoding="utf-8"))
96     if wallet.get("network") not in API_ROOTS:
97         raise ValueError("unsupported network in wallet")
98     return wallet
99
100
101 def list_utxos(wallet: dict[str, object]) -> list[dict[str, object]]:
102     root = API_ROOTS[str(wallet["network"])]
103     raw = api_request(f"{root}/address/{wallet['address']}/utxo")
104     return json.loads(raw)
105
106
107 def build_signed_transaction(
108     wallet: dict[str, object],
109     utxo: dict[str, object],
110     destination_program: bytes,
111     send_value: int,
112     fee_rate: int,
113 ) -> tuple[Transaction, int]:
114     input_value = int(utxo["value"])
115     destination_script = p2wpkh_script(destination_program)
116     change_script = bytes.fromhex(str(wallet["p2wpkh_program"]))
117     change_script = p2wpkh_script(change_script)
118
119     # A 1-input/2-output P2WPKH transaction is normally 141 vbytes. Sign once
120     # with a provisional fee, then recompute from the exact DER signature size.
121     provisional_fee = 141 * fee_rate
122     change_value = input_value - send_value - provisional_fee
123     if send_value < 294 or change_value < 294:
124         raise ValueError("send value or change would be below the P2WPKH dust threshold")
125     tx = Transaction(
126         version=2,
127         inputs=[TxInput(str(utxo["txid"]), int(utxo["vout"]), b"", 0xFFFFFFFF)],
128         outputs=[TxOutput(send_value, destination_script), TxOutput(change_value, change_script)],
129         locktime=0,
130         segwit=True,
131     )
132     secret = int(str(wallet["private_key_hex"]), 16)
133     pubkey = bytes.fromhex(str(wallet["public_key_compressed"]))
134     script_code = b"\x76\xa9\x14" + bytes.fromhex(str(wallet["p2wpkh_program"])) + b"\x88\xac"
135
136 def sign() -> None:
137     digest = bip143_sighash_all(tx, 0, script_code, input_value)
138     r, s = ecdsa_sign(secret, digest)
139     tx.inputs[0].witness = [der_encode_signature(r, s) + b"\x01", pubkey]
140

```



```

141     sign()
142     exact_fee = tx.vsize * fee_rate
143     tx.outputs[1].value = input_value - send_value - exact_fee
144     if tx.outputs[1].value < 294:
145         raise ValueError("exact change would be below dust")
146     sign()
147     return tx, exact_fee
148
149
150 def command_new(args: argparse.Namespace) -> None:
151     wallet = create_wallet(Path(args.wallet), args.network)
152     print(json.dumps({"network": wallet["network"], "address": wallet["address"], "wallet": args.wallet}, indent=2))
153
154
155 def command_status(args: argparse.Namespace) -> None:
156     wallet = load_wallet(Path(args.wallet))
157     utxos = list_utxos(wallet)
158     print(json.dumps({"network": wallet["network"], "address": wallet["address"], "utxos": utxos}, indent=2))
159
160
161 def command_send(args: argparse.Namespace) -> None:
162     wallet_path = Path(args.wallet)
163     wallet = load_wallet(wallet_path)
164     hrp, version, program = __import__("bitcoin_codec").decode_segwit_address(args.destination)
165     if hrp != "tb" or version != 0 or len(program) != 20:
166         raise ValueError("destination must be a tbiq P2WPKH address")
167     utxos = [
168         item
169         for item in list_utxos(wallet)
170         if args.allow_unconfirmed or item.get("status", {}).get("confirmed")
171     ]
172     if not utxos:
173         requirement = "UTXO" if args.allow_unconfirmed else "confirmed UTXO"
174         raise RuntimeError(f"no {requirement} is available for this wallet")
175     utxo = max(utxos, key=lambda item: int(item["value"]))
176     tx, fee = build_signed_transaction(wallet, utxo, program, args.amount, args.fee_rate)
177     raw_hex = tx.serialize().hex()
178     output = {
179         "network": wallet["network"],
180         "source_address": wallet["address"],
181         "destination_address": args.destination,
182         "input": utxo,
183         "send_value_satoshi": args.amount,
184         "change_value_satoshi": tx.outputs[1].value,
185         "fee_satoshi": fee,
186         "fee_rate_sat_vbyte": args.fee_rate,
187         "txid_local": tx.txid,
188         "wtxid_local": tx.wtxid,
189         "vsize": tx.vsize,
190         "weight": tx.weight,
191         "raw_transaction": raw_hex,
192     }
193     if args.broadcast:
194         root = API_ROOTS[str(wallet["network"])]
195         returned_txid = api_request(f"{root}/tx", raw_hex.encode("ascii")).decode("ascii").strip()
196         if returned_txid != tx.txid:
197             raise RuntimeError("broadcast service returned unexpected txid {returned_txid}")
198         output["broadcast_txid"] = returned_txid
199         output["explorer_url"] = f"https://mempool.space/{wallet['network']}/tx/{returned_txid}"
200         output["broadcast_unix"] = int(time.time())

```

```

201 out_path = Path(args.output)
202 out_path.parent.mkdir(parents=True, exist_ok=True)
203 out_path.write_text(json.dumps(output, ensure_ascii=False, indent=2), encoding="utf-8")
204 print(json.dumps(output, ensure_ascii=False, indent=2))
205
206
207 def parser() -> argparse.ArgumentParser:
208     result = argparse.ArgumentParser(description=__doc__)
209     subparsers = result.add_subparsers(dest="command", required=True)
210     new = subparsers.add_parser("new", help="generate a test-network-only wallet")
211     new.add_argument("--network", choices=sorted(API_ROOTS), default="signet")
212     new.add_argument("--wallet", default="data/sender_wallet.json")
213     new.set_defaults(func=command_new)
214     status = subparsers.add_parser("status", help="query wallet UTXOs")
215     status.add_argument("--wallet", default="data/sender_wallet.json")
216     status.set_defaults(func=command_status)
217     send = subparsers.add_parser("send", help="build and optionally broadcast a P2WPKH transaction")
218     send.add_argument("--wallet", default="data/sender_wallet.json")
219     send.add_argument("--destination", required=True)
220     send.add_argument("--amount", type=int, default=10_000, help="satoshi sent to destination")
221     send.add_argument("--fee-rate", type=int, default=2, help="satoshi per virtual byte")
222     send.add_argument("--output", default="output/broadcast.json")
223     send.add_argument("--broadcast", action="store_true")
224     send.add_argument("--allow-unconfirmed", action="store_true", help="allow spending an unconfirmed parent")
225     send.set_defaults(func=command_send)
226     return result
227
228
229 if __name__ == "__main__":
230     arguments = parser().parse_args()
231     arguments.func(arguments)

```

B.3 claim_op_true_utxos.py

```

1  """Spend explicitly public P2WSH(OP_TRUE) Testnet4 outputs into the course wallet.
2
3  The locking script is witness-v0 SHA256(0x51), where 0x51 is OP_TRUE. Supplying
4  0x51 as the witness script is therefore the complete, publicly authorized
5  unlocking condition. No third-party private key or signature is involved.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 from pathlib import Path
13 import time
14
15 from bitcoin_codec import Transaction, TxInput, TxOutput, p2wpkh_script, sha256
16 from wallet_tool import API_ROOTS, api_request, load_wallet
17
18
19 WITNESS_SCRIPT = b"\x51" # OP_TRUE
20 WITNESS_PROGRAM = sha256(WITNESS_SCRIPT)
21
22
23 def main() -> None:

```

```

24 parser = argparse.ArgumentParser(description=__doc__)
25 parser.add_argument("--wallet", default="data/testnet4_sender_wallet.json")
26 parser.add_argument("--inputs", type=int, default=5)
27 parser.add_argument("--output-value", type=int, default=1000)
28 parser.add_argument("--output", default="output/testnet4_op_true_funding.json")
29 parser.add_argument("--broadcast", action="store_true")
30 args = parser.parse_args()
31
32 wallet = load_wallet(Path(args.wallet))
33 if wallet["network"] != "testnet4":
34     raise ValueError("the public OP_TRUE outputs used here are on Testnet4")
35 root = API_ROOTS["testnet4"]
36 op_true_address = __import__("bitcoin_codec").segwit_address(WITNESS_PROGRAM)
37 utxos = json.loads(api_request(f"{root}/address/{op_true_address}/utxo"))
38 confirmed = [item for item in utxos if item.get("status", {}).get("confirmed")]
39 if len(confirmed) < args.inputs:
40     raise RuntimeError(f"need {args.inputs} confirmed OP_TRUE UTXOs, found {len(confirmed)}")
41 selected = confirmed[: args.inputs]
42 inputs = [TxInput(item["txid"], int(item["vout"]), b"", 0xFFFFFFFF, [WITNESS_SCRIPT]) for item in selected]
43 destination = p2wph_script(bytes.fromhex(str(wallet["p2wph_program"])))
44 tx = Transaction(2, inputs, [TxOutput(args.output_value, destination)], 0, segwit=True)
45 input_total = sum(int(item["value"]) for item in selected)
46 fee = input_total - args.output_value
47 if fee <= 0:
48     raise ValueError("output value leaves no transaction fee")
49 result: dict[str, object] = {
50     "network": "testnet4",
51     "authorization": "P2WSH witness program SHA256(OP_TRUE); witnessScript=OP_TRUE",
52     "op_true_address": op_true_address,
53     "witness_script_hex": WITNESS_SCRIPT.hex(),
54     "witness_program_hex": WITNESS_PROGRAM.hex(),
55     "selected_utxos": selected,
56     "input_total_satoshi": input_total,
57     "destination_address": wallet["address"],
58     "output_value_satoshi": args.output_value,
59     "fee_satoshi": fee,
60     "fee_rate_sat_vbyte": fee / tx.vsize,
61     "txid_local": tx.txid,
62     "wtxid_local": tx.wtxid,
63     "size": tx.total_size,
64     "weight": tx.weight,
65     "vsize": tx.vsize,
66     "raw_transaction": tx.serialize().hex(),
67 }
68 if args.broadcast:
69     returned = api_request(f"{root}/tx", tx.serialize().hex().encode("ascii")).decode("ascii").strip()
70     if returned != tx.txid:
71         raise RuntimeError(f"broadcast service returned unexpected txid {returned}")
72     result.update(
73         {
74             "broadcast_txid": returned,
75             "broadcast_unix": int(time.time()),
76             "explorer_url": f"https://mempool.space/testnet4/tx/{returned}",
77         }
78     )
79 output = Path(args.output)
80 output.parent.mkdir(parents=True, exist_ok=True)
81 output.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
82 print(json.dumps(result, ensure_ascii=False, indent=2))
83

```

```
84
85 if __name__ == "__main__":
86     main()
```

B.4 chain_analyzer.py

```
1  """Fetch and independently audit one Bitcoin transaction and one whole block."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  from datetime import datetime, timezone
8  import json
9  from pathlib import Path
10 import shutil
11 import subprocess
12 import time
13 import urllib.error
14 import urllib.request
15
16 from bitcoin_codec import (
17     Block,
18     Span,
19     parse_block,
20     parse_transaction,
21     verify_p2wpkh_input,
22     verify_p2wsh_op_true_input,
23 )
24
25
26 API_ROOTS = {
27     "mutinynet": "https://mutinynet.com/api",
28     "signet": "https://mempool.space/signet/api",
29     "testnet": "https://mempool.space/testnet/api",
30     "testnet4": "https://mempool.space/testnet4/api",
31 }
32
33
34 def api_get(root: str, path: str) -> bytes:
35     curl = shutil.which("curl.exe") or shutil.which("curl")
36     if curl:
37         result = subprocess.run(
38             [curl, "-sS", "-L", "--retry", "3", "--max-time", "60", root + path],
39             capture_output=True,
40             check=False,
41         )
42         if result.returncode == 0:
43             return result.stdout
44         raise RuntimeError(
45             f"curl failed for {path}: {result.stderr.decode(errors='replace').strip()}"
46         )
47     last_error: Exception | None = None
48     for attempt in range(1, 4):
49         request = urllib.request.Request(root + path, headers={"User-Agent": "coursework-byte-auditor/1.0"})
50         try:
51             with urllib.request.urlopen(request, timeout=60) as response:
```

```

52         return response.read()
53     except urllib.error.HTTPError as exc:
54         body = exc.read().decode("utf-8", errors="replace")
55         raise RuntimeError(f"HTTP {exc.code} for {path}: {body}") from exc
56     except (urllib.error.URLError, TimeoutError) as exc:
57         last_error = exc
58         if attempt < 3:
59             time.sleep(attempt)
60     raise RuntimeError(f"network request failed after 3 attempts for {path}: {last_error}")
61
62
63 def write_bit_map(path: Path, raw: bytes, spans: list[Span]) -> None:
64     labels = ["UNCLAIMED"] * len(raw)
65     for span in spans:
66         if not (0 <= span.start <= span.end <= len(raw)):
67             raise ValueError(f"span outside byte array: {span}")
68         for offset in range(span.start, span.end):
69             if labels[offset] != "UNCLAIMED":
70                 raise ValueError(f"overlapping span at byte {offset}: {labels[offset]} and {span.name}")
71             labels[offset] = span.name
72     missing = [index for index, label in enumerate(labels) if label == "UNCLAIMED"]
73     if missing:
74         raise ValueError(f"{len(missing)} bytes were not assigned to a protocol field")
75     with path.open("w", encoding="utf-8", newline="\n") as handle:
76         handle.write("offset_dec offset_hex byte_hex bits      field\n")
77         for offset, value in enumerate(raw):
78             handle.write(f"{offset:10d} 0x{offset:08x} {value:02x}      {value:08b} {labels[offset]}\n")
79
80
81 def write_spans(path: Path, raw: bytes, spans: list[Span]) -> None:
82     with path.open("w", encoding="utf-8", newline="") as handle:
83         writer = csv.writer(handle)
84         writer.writerow(["start", "end_exclusive", "length", "field", "wire_hex", "interpreted_value"])
85         for span in spans:
86             writer.writerow([span.start, span.end, span.end - span.start, span.name, raw[span.start:span.end].hex(),
87                             span.value])
88
89 def block_summary(block: Block) -> dict[str, object]:
90     target_ok = int.from_bytes(bytes.fromhex(block.block_hash), "big") <= block.target
91     return {
92         "block_hash_computed": block.block_hash,
93         "version": block.version,
94         "previous_hash": block.previous_hash,
95         "merkle_root_header": block.merkle_root,
96         "merkle_root_computed": block.computed_merkle_root,
97         "merkle_root_valid": block.merkle_root == block.computed_merkle_root,
98         "timestamp": block.timestamp,
99         "timestamp_utc": datetime.fromtimestamp(block.timestamp, timezone.utc).isoformat(),
100        "bits_hex": f"0x{block.bits:08x}",
101        "target_hex": f"{block.target:064x}",
102        "nonce": block.nonce,
103        "proof_of_work_valid": target_ok,
104        "transaction_count": len(block.transactions),
105        "block_size_bytes": len(block.raw),
106        "all_bytes_accounted": sum(span.end - span.start for span in block.spans) == len(block.raw),
107    }
108
109
110 def analyze(network: str, txid: str, block_hash: str | None, output_dir: Path) -> dict[str, object]:

```

```

111 root = API_ROOTS[network]
112 output_dir.mkdir(parents=True, exist_ok=True)
113 tx_raw = bytes.fromhex(api_get(root, f"/tx/{txid}/hex").decode("ascii").strip())
114 tx, consumed = parse_transaction(tx_raw)
115 if consumed != len(tx_raw) or tx.txid != txid:
116     raise ValueError("transaction parse or txid cross-check failed")
117 tx_info = json.loads(api_get(root, f"/tx/{txid}"))
118 if block_hash is None:
119     block_hash = tx_info.get("status", {}).get("block_hash")
120 if not block_hash:
121     raise ValueError("transaction is unconfirmed; pass --block-hash to audit a complete block")
122
123 script_audits: list[dict[str, object]] = []
124 for index, txin in enumerate(tx.inputs):
125     prev_tx = json.loads(api_get(root, f"/tx/{txin.prev_txid}"))
126     prevout = prev_tx["vout"][txin.prev_vout]
127     prev_script = bytes.fromhex(prevout["scriptpubkey"])
128     if len(prev_script) == 22 and prev_script[:2] == b"\x00\x14":
129         result = verify_p2wpkh_input(tx, index, prev_script, int(prevout["value"]))
130     elif len(prev_script) == 34 and prev_script[:2] == b"\x00\x20" and txin.witness == [b"\x51"]:
131         result = verify_p2wsh_op_true_input(tx, index, prev_script)
132     else:
133         continue
134     if result:
135         result.update(
136             {
137                 "input_index": index,
138                 "prev_txid": txin.prev_txid,
139                 "prev_vout": txin.prev_vout,
140                 "prevout_value_satoshi": int(prevout["value"]),
141                 "prevout_script_pubkey": prev_script.hex(),
142             }
143         )
144         script_audits.append(result)
145
146 block_raw = api_get(root, f"/block/{block_hash}/raw")
147 block = parse_block(block_raw)
148 summary = block_summary(block)
149 if block.block_hash != block_hash:
150     raise ValueError("computed block hash does not match requested hash")
151 if not summary["merkle_root_valid"] or not summary["proof_of_work_valid"]:
152     raise ValueError("block cryptographic validation failed")
153
154 transaction_summary = {
155     "network": network,
156     "txid_requested": txid,
157     "txid_computed": tx.txid,
158     "wtxid_computed": tx.wtxid,
159     "segwit": tx.segwit,
160     "version": tx.version,
161     "input_count": len(tx.inputs),
162     "output_count": len(tx.outputs),
163     "locktime": tx.locktime,
164     "total_size_bytes": tx.total_size,
165     "stripped_size_bytes": tx.stripped_size,
166     "weight_units": tx.weight,
167     "virtual_size_bytes": tx.vsize,
168     "all_bytes_accounted": sum(span.end - span.start for span in tx.spans) == len(tx_raw),
169     "status_from_api": tx_info.get("status"),
170     "outputs": [

```

```

171         {"index": index, "value_satoshi": item.value, "script_pubkey": item.script_pubkey.hex()}
172         for index, item in enumerate(tx.outputs)
173     ],
174 }
175
176 (output_dir / "transaction.raw.hex").write_text(tx_raw.hex() + "\n", encoding="ascii")
177 (output_dir / "block.raw.hex").write_text(block_raw.hex() + "\n", encoding="ascii")
178 (output_dir / "transaction_summary.json").write_text(
179     json.dumps(transaction_summary, ensure_ascii=False, indent=2), encoding="utf-8"
180 )
181 (output_dir / "script_audit.json").write_text(
182     json.dumps(script_audits, ensure_ascii=False, indent=2), encoding="utf-8"
183 )
184 (output_dir / "block_summary.json").write_text(
185     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
186 )
187 write_bit_map(output_dir / "transaction_bits.txt", tx_raw, tx.spans)
188 write_bit_map(output_dir / "block_bits.txt", block_raw, block.spans)
189 write_spans(output_dir / "transaction_fields.csv", tx_raw, tx.spans)
190 write_spans(output_dir / "block_fields.csv", block_raw, block.spans)
191 with (output_dir / "block_transaction_index.csv").open("w", encoding="utf-8", newline="") as handle:
192     writer = csv.writer(handle)
193     writer.writerow(["index", "txid", "wtxid", "segwit", "size", "stripped_size", "weight", "vsize"])
194     for index, item in enumerate(block.transactions):
195         writer.writerow([index, item.txid, item.wtxid, item.segwit, item.total_size, item.stripped_size, item.
196             weight, item.vsize])
197
198 final = {"transaction": transaction_summary, "script_audits": script_audits, "block": summary}
199 print(json.dumps(final, ensure_ascii=False, indent=2))
200 return final
201
202 def parser() -> argparse.ArgumentParser:
203     result = argparse.ArgumentParser(description=__doc__)
204     result.add_argument("--network", choices=sorted(API_ROOTS), default="signet")
205     result.add_argument("--txid", required=True)
206     result.add_argument("--block-hash")
207     result.add_argument("--output", default="output")
208     return result
209
210
211 if __name__ == "__main__":
212     args = parser().parse_args()
213     analyze(args.network, args.txid, args.block_hash, Path(args.output))

```

B.5 visualize_results.py

```

1  """Render report-ready figures directly from the analyzer's JSON/CSV output."""
2
3  from __future__ import annotations
4
5  import csv
6  import json
7  from pathlib import Path
8
9  import matplotlib.pyplot as plt
10 from matplotlib.font_manager import FontProperties

```

```

11 from matplotlib.patches import FancyBboxPatch, Rectangle
12
13
14 PROJECT_ROOT = Path(__file__).resolve().parent.parent
15 OUTPUT = PROJECT_ROOT / "output" / "testnet4_signed_analysis"
16 FIGURES = PROJECT_ROOT / "output" / "figures"
17 FONT_PATH = Path(r"C:\Windows\Fonts\msyh.ttc")
18 CN = FontProperties(fname=str(FONT_PATH)) if FONT_PATH.exists() else None
19 MONO = FontProperties(family="DejaVu Sans Mono")
20
21 COLORS = {
22     "version": "#355070",
23     "segwit": "#6d597a",
24     "input": "#b56576",
25     "output": "#e56b6f",
26     "witness": "#eac8b",
27     "locktime": "#4d908e",
28 }
29
30
31 def group_for(field: str) -> str:
32     if field.endswith("version"):
33         return "version"
34     if field.endswith("marker") or field.endswith("flag"):
35         return "segwit"
36     if ".vin[" in field and ".witness" not in field:
37         return "input"
38     if ".vout[" in field:
39         return "output"
40     if ".witness" in field:
41         return "witness"
42     return "locktime"
43
44
45 def transaction_layout() -> None:
46     summary = json.loads((OUTPUT / "transaction_summary.json").read_text(encoding="utf-8"))
47     rows = list(csv.DictReader((OUTPUT / "transaction_fields.csv").open(encoding="utf-8")))
48     intervals: list[tuple[int, int, str]] = []
49     for row in rows:
50         start, end = int(row["start"]), int(row["end_exclusive"])
51         group = group_for(row["field"])
52         if intervals and intervals[-1][2] == group and intervals[-1][1] == start:
53             intervals[-1] = (intervals[-1][0], end, group)
54         else:
55             intervals.append((start, end, group))
56     total = max(end for _, end, _ in intervals)
57     fig, ax = plt.subplots(figsize=(12, 3.3))
58     for start, end, group in intervals:
59         ax.add_patch(Rectangle((start, 0), end - start, 1, color=COLORS[group], ec="white", lw=0.8))
60         if end - start >= 6:
61             ax.text((start + end) / 2, 0.5, f"{group}\n{end-start} B", ha="center", va="center", color="white",
62                     fontsize=8)
63     ax.set_xlim(0, total)
64     ax.set_ylim(0, 1)
65     ax.set_yticks([])
66     ax.set_xlabel(f"线协议字节偏移 (总计 {summary['total_size_bytes']} B)", fontproperties=CN)
67     ax.set_title("SegWit 交易逐字节布局", fontproperties=CN, fontsize=15, pad=14)
68     ax.spines[["top", "left", "right"]].set_visible(False)
69     fig.tight_layout()
70     fig.savefig(FIGURES / "transaction_byte_layout.png", dpi=220, bbox_inches="tight")

```



```

70 plt.close(fig)
71
72
73 def script_trace() -> None:
74     audit = json.loads((OUTPUT / "script_audit.json").read_text(encoding="utf-8"))[0]
75     trace = audit["trace"]
76     fig, ax = plt.subplots(figsize=(11, 6.8))
77     ax.set_xlim(0, 10)
78     ax.set_ylim(-1.05, len(trace) - 0.3)
79     ax.axis("off")
80     for index, step in enumerate(trace):
81         y = len(trace) - 1 - index
82         depth = len(step["stack"])
83         box = FancyBboxPatch((0.35, y - 0.31), 2.3, 0.62, boxstyle="round,pad=0.03", fc="#355070", ec="none")
84         ax.add_patch(box)
85         ax.text(1.5, y, step["operation"], ha="center", va="center", color="white", fontproperties=MONO, fontsize
            =10)
86         ax.text(2.95, y + 0.10, step["detail"], ha="left", va="center", fontproperties=MONO, fontsize=8.5)
87         ax.text(2.95, y - 0.16, f"stack depth = {depth}", ha="left", va="center", color="#555555", fontproperties=
            MONO, fontsize=8)
88         for stack_index in range(depth):
89             ax.add_patch(Rectangle((8.9 - stack_index * 0.58, y - 0.22), 0.48, 0.44, fc="#eaac8b", ec="#9b4d5b", lw
                =0.7))
90             if index < len(trace) - 1:
91                 ax.annotate("", xy=(1.5, y - 0.61), xytext=(1.5, y - 0.39), arrowprops={"arrowstyle": "->", "color": "
                    #777777"})
92     ax.set_title("P2WPKH Script 栈执行轨迹", fontproperties=CN, fontsize=15, pad=12)
93     ax.text(0.35, -0.75, "最终栈 = [01], ECDSA = valid, CleanStack = true", fontproperties=CN, fontsize=10, color="
        #176b4d")
94     fig.tight_layout()
95     fig.savefig(FIGURES / "script_stack_trace.png", dpi=220, bbox_inches="tight")
96     plt.close(fig)
97
98
99 def validation_summary() -> None:
100     tx = json.loads((OUTPUT / "transaction_summary.json").read_text(encoding="utf-8"))
101     block = json.loads((OUTPUT / "block_summary.json").read_text(encoding="utf-8"))
102     audit = json.loads((OUTPUT / "script_audit.json").read_text(encoding="utf-8"))[0]
103     fig, ax = plt.subplots(figsize=(12, 7))
104     fig.patch.set_facecolor("#0f1419")
105     ax.set_facecolor("#0f1419")
106     ax.axis("off")
107     lines = [
108         ("BITCOIN TESTNET4 BYTE AUDIT", "#7ee787", 18),
109         (f"$ python chain_analyzer.py --network {tx['network']} --txid {tx['txid_computed'][:8]}...", "#b6beca", 10)
110     ],
111     ("", "white", 8),
112     (f"[OK] txid {tx['txid_computed']}", "#7ee787", 10),
113     (f"[OK] bytes {tx['total_size_bytes']} / {tx['total_size_bytes']} accounted", "#7ee787", 10),
114     (f"[OK] BIP143 {audit['sighash_digest']}", "#7ee787", 10),
115     (f"[OK] Script ECDSA={audit['ecdsa_valid']} low-S={audit['low_s']} CleanStack={audit['
        cleanstack_success']}", "#7ee787", 10),
116     ("", "white", 8),
117     (f"[OK] block {block['block_hash_computed']}", "#7ee787", 10),
118     (f"[OK] full parse {block['block_size_bytes']:,} bytes, {block['transaction_count']} transactions", "#7ee787
        ", 10),
119     (f"[OK] Merkle {block['merkle_root_computed']}", "#7ee787", 10),
120     (f"[OK] PoW hash <= {block['target_hex']}", "#7ee787", 10),
121     ("", "white", 8),
122     ("RESULT: ALL INDEPENDENT CHECKS PASSED", "#ffcc66", 13),

```

```

122 ]
123 y = 0.93
124 for text, color, size in lines:
125     ax.text(0.045, y, text, transform=ax.transAxes, color=color, fontproperties=MONO, fontsize=size, va="top")
126     y -= 0.064 if text else 0.035
127 fig.savefig(FIGURES / "runtime_validation.png", dpi=220, bbox_inches="tight", facecolor=fig.get_facecolor())
128 plt.close(fig)
129
130
131 def main() -> None:
132     FIGURES.mkdir(parents=True, exist_ok=True)
133     transaction_layout()
134     script_trace()
135     validation_summary()
136     print(f"rendered 3 figures in {FIGURES}")
137
138
139 if __name__ == "__main__":
140     main()

```

B.6 test_bitcoin_codec.py

```

1  """Deterministic regression tests for the coursework Bitcoin implementation."""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7  import tempfile
8  import unittest
9
10 from bitcoin_codec import (
11     N,
12     ByteReader,
13     decode_segwit_address,
14     der_decode_signature,
15     der_encode_signature,
16     ecdsa_sign,
17     ecdsa_verify,
18     encode_pubkey,
19     hash160,
20     parse_block,
21     parse_transaction,
22     scalar_mult,
23     segwit_address,
24     sha256,
25     verify_p2wpkh_input,
26     verify_p2wsh_op_true_input,
27 )
28 from chain_analyzer import write_bit_map
29
30
31 PROJECT_ROOT = Path(__file__).resolve().parent.parent
32 SAMPLE = PROJECT_ROOT / "output" / "sample_confirmed"
33
34
35 class BitcoinCodecTests(unittest.TestCase):

```

```

36 def test_secp256k1_generator_and_ecdsa(self) -> None:
37     public_key = encode_pubkey(scalar_mult(1))
38     self.assertEqual(
39         public_key.hex(),
40         "0279be667ef9dcbbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798",
41     )
42     digest = sha256(b"network-security-coursework")
43     r, s = ecdsa_sign(1, digest)
44     self.assertLessEqual(s, N // 2)
45     self.assertTrue(ecdsa_verify(public_key, digest, r, s))
46     self.assertFalse(ecdsa_verify(public_key, sha256(b"mutated"), r, s))
47     self.assertEqual(der_decode_signature(der_encode_signature(r, s)), (r, s))
48
49 def test_bech32_round_trip(self) -> None:
50     program = hash160(encode_pubkey(scalar_mult(1)))
51     address = segwit_address(program)
52     self.assertEqual(address, "tb1qw508d6qejxtdg4y5r3zarvary0c5xw7kxpjzsx")
53     self.assertEqual(decode_segwit_address(address), ("tb", 0, program))
54
55 def test_compact_size_rejects_non_minimal_encoding(self) -> None:
56     with self.assertRaisesRegex(ValueError, "non-minimal"):
57         ByteReader(b"\xfd\xfc\x00").varint("bad")
58
59 @unittest.skipUnless(SAMPLE.exists(), "run chain_analyzer.py to obtain the confirmed sample")
60 def test_confirmed_transaction_and_script(self) -> None:
61     raw = bytes.fromhex((SAMPLE / "transaction.raw.hex").read_text(encoding="ascii"))
62     tx, consumed = parse_transaction(raw)
63     self.assertEqual(consumed, len(raw))
64     self.assertEqual(tx.serialize(), raw)
65     self.assertEqual(tx.txid, "d083bf3e9d9eb467f9b1577b5d4b85174cce9d94f4c33f6d872d39e5d08c0689")
66     self.assertEqual(tx.wtxid, "7742b336d2d4825c2e5c5e68828c6eccd08a295504cc503c361232a7af569e97")
67     result = verify_p2wpkh_input(
68         tx,
69         0,
70         bytes.fromhex("0014fbc7a1586fc3c289af9d07dcf249fbab804e7a50"),
71         778_123,
72     )
73     self.assertTrue(result["ecdsa_valid"])
74     self.assertTrue(result["cleanstack_success"])
75     self.assertEqual(len(result["trace"]), 6)
76
77 @unittest.skipUnless(SAMPLE.exists(), "run chain_analyzer.py to obtain the confirmed sample")
78 def test_complete_block_and_byte_accounting(self) -> None:
79     raw = bytes.fromhex((SAMPLE / "block.raw.hex").read_text(encoding="ascii"))
80     block = parse_block(raw)
81     summary = json.loads((SAMPLE / "block_summary.json").read_text(encoding="utf-8"))
82     self.assertEqual(block.block_hash, summary["block_hash_computed"])
83     self.assertEqual(block.computed_merkle_root, block.merkle_root)
84     self.assertLessEqual(int(block.block_hash, 16), block.target)
85     self.assertEqual(len(block.transactions), 41)
86     self.assertEqual(sum(span.end - span.start for span in block.spans), len(raw))
87     with tempfile.TemporaryDirectory() as directory:
88         bit_map = Path(directory) / "bits.txt"
89         write_bit_map(bit_map, raw, block.spans)
90         self.assertEqual(len(bit_map.read_text(encoding="utf-8").splitlines()), len(raw) + 1)
91
92 def test_p2wsh_op_true_authorization(self) -> None:
93     from bitcoin_codec import Transaction, TxInput, TxOutput
94
95     tx = Transaction(

```

```
96         2,  
97         [TxInput("00" * 32, 0, b"", 0xFFFFFFFF, [b"\x51"])],  
98         [TxOutput(1, b"\x51")],  
99         0,  
100        segwit=True,  
101    )  
102    previous_script = b"\x00\x20" + sha256(b"\x51")  
103    result = verify_p2wsh_op_true_input(tx, 0, previous_script)  
104    self.assertTrue(result["witness_program_matches"])  
105    self.assertTrue(result["cleanstack_success"])  
106  
107  
108    if __name__ == "__main__":  
109        unittest.main(verbosity=2)
```

参考文献

- [1] Fabian Jahr. *BIP 94: Testnet 4*. Bitcoin BIPs, 2024. <https://github.com/bitcoin/bips/blob/master/bip-0094.mediawiki>. 访问日期: 2026-07-20.
- [2] Eric Lombrozo, Johnson Lau, Pieter Wuille. *BIP 141: Segregated Witness (Consensus layer)*. Bitcoin BIPs, 2015. <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>. 访问日期: 2026-07-20.
- [3] Johnson Lau, Pieter Wuille. *BIP 143: Transaction Signature Verification for Version 0 Witness Program*. Bitcoin BIPs, 2016. <https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki>. 访问日期: 2026-07-20.
- [4] Pieter Wuille, Greg Maxwell. *BIP 173: Base32 address format for native v0-16 witness outputs*. Bitcoin BIPs, 2017. <https://github.com/bitcoin/bips/blob/master/bip-0173.mediawiki>. 访问日期: 2026-07-20.
- [5] Karl-Johan Alm, Anthony Towns. *BIP 325: Signet*. Bitcoin BIPs, 2019. <https://github.com/bitcoin/bips/blob/master/bip-0325.mediawiki>. 访问日期: 2026-07-20.
- [6] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. <https://bitcoin.org/bitcoin.pdf>. 访问日期: 2026-07-20.
- [7] Bitcoin.org. *Bitcoin Developer Reference: Transactions and Block Chain*. <https://developer.bitcoin.org/reference/transactions.html>; https://developer.bitcoin.org/reference/block_chain.html. 访问日期: 2026-07-20.
- [8] The Mempool Open Source Project. *mempool REST API Documentation*. <https://github.com/mempool/mempool/blob/master/docs/api/rest.md>. 访问日期: 2026-07-20.
- [9] T. Pornin. *RFC 6979: Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)*. IETF, 2013. <https://www.rfc-editor.org/rfc/rfc6979>. 访问日期: 2026-07-20.